Cinema Knowledge Graph: Full Pipeline Report

Data Acquisition, KB Alignment, Reasoning, KGE & SPARQL RAG

Web Mining & Semantics

March 2026

Abstract

This report outlines the end-to-end development of a domain-specific Knowledge Graph (KG) centered on cinema. The pipeline progresses through four distinct phases: extracting unstructured text from Wikipedia, aligning and expanding entities via Wikidata, applying logical reasoning alongside Knowledge Graph Embedding (KGE) training, and deploying a Retrieval-Augmented Generation (RAG) system. Ultimately, the project demonstrates both the potential and the severe limitations of constraining a local Large Language Model (Mistral) to generate SPARQL queries against a custom RDF graph.

1 Data Acquisition & Information Extraction

1.1 Domain, Seed URLs, and Crawler Design

The cinema domain offers a dense, heavily interlinked network of directors, actors, films, and awards, which makes it a perfect theme. To seed the dataset, eight Wikipedia pages were selected, dividing the focus equally between acclaimed directors (e.g., Stanley Kubrick, Christopher Nolan) and iconic films (e.g., *The Godfather, Inception*). Wikipedia's high information density and standardized formatting make it an ideal target for NLP pipelines.

Rather than relying on standard `requests`, the crawler leverages `httpx` to allow for future asynchronous scaling. Furthermore, it utilizes `trafilatura` to automatically detect and isolate the main article body. This avoids the need to write brittle, site-specific regex rules to strip away navigation bars and footers.

```python
import httpx
import trafilatura
import time

# A custom User-Agent prevents silent 403 blocks from Wikipedia
HEADERS = {"User-Agent": "CinemaKnowledgeGraphBot/1.0 (Student Project)"}

def fetch_and_clean(url):
    time.sleep(1) # Strict 1-second delay to adhere to web scraping ethics
    response = httpx.get(url, headers=HEADERS)
    if response.status_code == 200:
        clean_text = trafilatura.extract(response.text)
        return clean_text
    return None
```

Listing 1: Web Crawling and Boilerplate Removal

By enforcing a strict 1-second delay, the scraper respects Wikipedia's rate limits. All eight pages comfortably passed the initial 500-word quality filter, resulting in a total corpus of 141,442 words.

1.2 Cleaning Pipeline & Named Entity Recognition

Extracting structured data from the raw text required a robust NLP pipeline, which `spaCy` provided. I specifically opted for the transformer-based model (`en_core_web_trf`) instead of the lighter `sm` model. Early testing revealed that smaller models consistently failed to recognize multi-word film titles, whereas the transformer architecture successfully detected complex spans, identifying 1,984 film and art entities.

Table 1: Entity distribution across the extracted corpus

Label	Count	% of total
PERSON	3,739	32.5%
DATE	3,703	32.2%
WORK OF ART	1,984	17.2%
ORG	1,708	14.8%
GPE	375	3.3%
Total	**11,509**	**100%**

Dependency parsing drove the relation extraction step. By anchoring on verbs (`POS == "VERB"`) and tracking subject (`nsubj`) and object (`dobj`) dependencies, the script heuristically constructs relational triples without needing a pre-trained relation extraction model.

```python
doc = nlp(sentence_text)
for token in doc:
    if token.pos_ == "VERB":
        # Extract the subjects and objects directly attached to the verb
        subjects = [w for w in token.lefts if w.dep_ in ("nsubj", "nsubjpass")]
        objects = [w for w in token.rights if w.dep_ in ("dobj", "pobj", "attr")]

        if subjects and objects:
            subj_entity = identify_entity(subjects[0])
            obj_entity = identify_entity(objects[0])
            if subj_entity and obj_entity:
                triples.append((subj_entity, token.lemma_, obj_entity))
```

Listing 2: Dependency Parsing for Relation Extraction

1.3 Ambiguity Cases in Extraction

Reviewing the dependency parser's output highlighted several structural limitations inherent to this approach. Even with a highly accurate transformer model, semantic ambiguities emerged:

1. **Pronoun Resolution:** The pipeline routinely extracted triples such as `(he) --[become]--> (one)`. Without coreference resolution, the parser cannot map pronouns back to their named entities.

2. **Spurious ORG Attachment:** A triple surfaced reading `(NASA) --[film]--> (scenes)`. The model correctly tagged NASA, but because the sentence used "film" as a verb, it blindly generated a link. This perfectly illustrates that syntactic validity does not guarantee semantic truth.

2 KB Construction & Alignment

2.1 RDF Modeling and Entity Alignment

Raw text strings alone do not constitute a Knowledge Graph; they must resolve to unique URIs. To achieve this, the pipeline queries the 6,194 candidate entities against the Wikidata Search API. Handling transient network failures required an exponential backoff loop.

I designed a strict confidence scoring heuristic: exact string matches (case-insensitive) received a 0.95 score, while partial matches returned by the API scored 0.70. The script only created `owl:sameAs` links for entities scoring ≥ 0.90.

```python
def align_entity_wikidata(entity_label):
    url = "https://www.wikidata.org/w/api.php"
    params = {"action": "wbsearchentities", "search": entity_label, "language": "en", "format": "json"}

    res = requests.get(url, params=params, headers=HEADERS).json() # Retries omitted

    if res.get('search'):
        match = res['search'][0]
        # Heavily reward exact string matches to prevent erroneous links
        confidence = 0.95 if match['label'].lower() == entity_label.lower() else 0.70
        return match['id'], match['label'], confidence
    return None, None, 0.0
```

Listing 3: Entity Alignment with Confidence Scoring

This cautious alignment strategy linked 4,403 entities, resulting in a solid 71.1% alignment rate.

2.2 Predicate Alignment & Expansion Strategy

Following entity resolution, the extracted local relations needed mapping to standard Wikidata properties (e.g., linking `:direct` to `wdt:P57`). Once aligned, federated SPARQL queries expanded the graph to meet the training volume requirements.

A predicate-controlled expansion strategy proved most effective here. The queries targeted seven specific cinema properties, explicitly restricting subjects to instances of film (`wd:Q11424`). This constraint maintained domain coherence, preventing the graph from bloating with generic geographic or corporate data.

```cpp
SELECT ?subject ?object ?objectLabel WHERE {
  # The Q11424 filter ensures we only fetch facts about films
  ?subject wdt:P31 wd:Q11424 .
  ?subject wdt:P161 ?object .  # Cast member property
  SERVICE wikibase:label { bd:serviceParam wikibase:language "en". }
} LIMIT 10000
```

Listing 4: Predicate-Controlled SPARQL Expansion

This phase successfully pushed the dataset to 99,106 triples and 66,864 entities.

3 Reasoning (SWRL)

3.1 Ontology Reasoning with OWLReady2

As a preliminary check, the reasoning logic was validated using a standard `family.owl` ontology via `OWLReady2`. By injecting a simple SWRL rule, the reasoner dynamically classified any person over 60 into an `oldPerson` category.

```python
from owlready2 import *

onto = get_ontology("family.owl").load()
with onto:
    class oldPerson(Person): pass

    rule = Imp()
    rule.set_as_rule("Person(?p), age(?p, ?age), greaterThan(?age, 60) -> oldPerson(?p)")

sync_reasoner_pellet(infer_property_values=True, infer_data_property_values=True)
```

Listing 5: SWRL Execution via OWLReady2

The Pellet reasoner executed this flawlessly, updating the ontology with the inferred classifications.

3.2 Cinema Domain Reasoning

Transitioning back to the custom cinema graph, I designed a Horn rule to infer professional networks. The logic, `Film(?f) ^ director(?f, ?d) ^ director(?f, ?d2) -> workedWith(?d, ?d2)`, generates new edges between co-directors. This highlights the primary advantage of logical inference: surfacing latent relationships without scraping additional data.

4 Knowledge Graph Embeddings (KGE)

4.1 Data Cleaning and Training Setup

Preparing the expanded graph for KGE training revealed a massive noise issue. Out of 99,106 expanded triples, roughly 99% consisted of Wikidata maintenance metadata (e.g., `schema:dateModified`). Training on this raw data would teach the model about Wikipedia's administrative habits rather than cinema. Filtering out everything except the seven core cinema predicates condensed the graph down to just 276 highly relevant triples.

```python
from pykeen.pipeline import pipeline

result = pipeline(
    training=train_factory,
    testing=test_factory,
    validation=valid_factory,
    model='TransE',
    training_kwargs=dict(num_epochs=500, batch_size=512),
    optimizer_kwargs=dict(lr=0.01),
    device='cuda'
)
print(f"TransE MRR: {result.metric_results.get_metric('mrr')}")
```

Listing 6: PyKEEN Pipeline Execution

4.2 Metrics and Clustering Analysis

Evaluating the KGE models involved comparing TransE alongside ComplEx. The results in Table 2 underscore the profound impact of dataset size on different model architectures.

Table 2: Link prediction results on the filtered test set

Model	MRR	Hits@1	Hits@3	Hits@10
TransE	0.1683	0.1250	0.1250	0.1875
ComplEx	0.0156	0.0000	0.0000	0.0000

TransE vastly outperformed ComplEx. Because ComplEx utilizes complex-valued vectors to capture asymmetric relations, its parameter count is essentially double that of TransE. Given a training set of only 220 triples, ComplEx collapsed under parameter overload. In contrast, TransE's simpler translational objective ($h + r \approx t$) managed to generalize effectively despite the extreme sparsity.

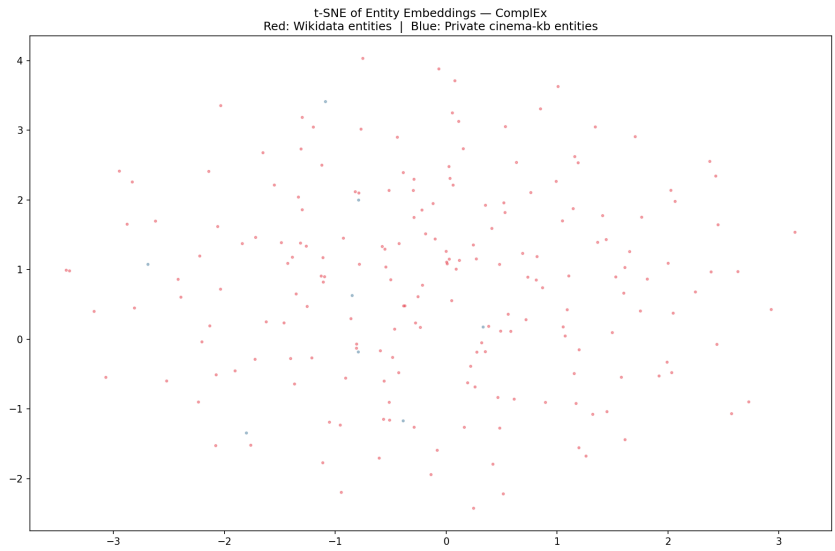

Figure 1: t-SNE projection of TransE embeddings. Without dense class relations, entities cluster primarily by relational proximity.

The resulting t-SNE plot (Figure 1) lacks distinct boundaries separating films from directors. Since the filtered triples consist almost entirely of "cast member" and "award received" edges, the embedding space organizes geometrically around collaborative proximity rather than sweeping ontological classes.

5 RAG over RDF/SPARQL

5.1 Prompt Architecture and Schema Linking

The pipeline culminates in a Retrieval-Augmented Generation (RAG) system powered by a local Mistral LLM via Ollama. Instead of allowing the LLM to generate plain text answers, the system forces it to act as a semantic parser, translating natural language into SPARQL 1.1.

Through iterative testing, the prompt engineering required heavy modifications to accommodate local graph structures. I implemented a strict ruleset that removed `@en` tags, enforced a custom namespace (`ns1:<wdt:>`), and actively prohibited hardcoded Wikidata IDs:

```
prompt = f"""You are an expert SPARQL generator. Convert the user QUESTION into
    a valid SPARQL 1.1 SELECT query.
Follow these rules strictly:
- Do NOT output any natural language, greetings, or explanations.
- Return ONLY the query wrapped in ```sparql ... ```
```

```
- Use the provided SCHEMA SUMMARY to find the correct prefixes and predicates.
- Use the prefix ‘PREFIX ns1: <wdt:>‘ instead of the standard ‘wdt:‘ URI.
- Query properties using ‘ns1:‘ (e.g., ‘ns1:P57‘ for director).
- Never append ‘@en‘ to string literals. Always use plain strings.
- Never hardcode Wikidata entity IDs (like wd:Q123) in the query. Always use
    variables (like ?entity) and match them to the user’s requested entity using
     rdfs:label "Entity Name".

SCHEMA SUMMARY: {schema}
QUESTION: {question}"""
```

Listing 7: Revised System Prompt

5.2 Evaluation: Baseline vs RAG (5 Questions)

To benchmark the prompt's effectiveness, the system was tested across five specific questions. The results expose deep inconsistencies in the LLM's adherence to the rules.

1. **Q: Who directed Inception?**
   Baseline: Answered correctly (Christopher Nolan).
   RAG: Successfully wrote SPARQL matching `rdfs:label "Inception"` with predicate `ns1:P57`. It returned `Q25191` (Nolan's Wikidata ID). This was a complete success.

2. **Q: Which films did Christopher Nolan direct?**
   Baseline: Provided an accurate list.
   RAG: The model hallucinated a DBpedia URI (`BIND(<http://dbpedia.org/resource/Christopher_Nolan> AS ?person)`) and used a flawed `OPTIONAL` block, which resulted in 9,347 generic rows being returned.

3. **Q: Who are the cast members of The Godfather?**
   Baseline: Provided an accurate list of actors.
   RAG: The model generated a syntactically valid query but utilized `ns1:P364` ("original language") instead of `P161` ("cast member"), causing it to return URIs for English and Italian rather than actors.

4. **Q: Which awards did Martin Scorsese receive?**
   Baseline: Listed correct awards.
   RAG: Ignored negative constraints entirely by hardcoding an ID (`BIND(wd:"Q56")`), causing a syntax crash. While the self-repair loop fixed the syntax, the LLM hallucinated predicates (`P272`), resulting in zero rows returned.

5. **Q: What is the country of origin of Pulp Fiction?**
   Baseline: Answered USA.
   RAG: Generated a structurally sound query but completely hallucinated a new predicate (`ns1:P910`), returning zero rows.

```
Question (or 'quit'): Who directed Inception?

--- Baseline (No RAG) ---
 The movie "Inception" was directed by Christopher Nolan. He is known for other famous films such as "The Dark Knight",
"Interstellar", and "Dunkirk".

--- SPARQL-generation RAG (Mistral + rdflib) ---

[SPARQL Query Used]
PREFIX ns1: <wdt:>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?director WHERE {
  ?movie rdfs:label "Inception" .
  ?movie ns1:P57 ?director .
}

[Repaired?] False

[Results]
director
http://www.wikidata.org/entity/Q25191
```

Figure 2: The RAG CLI demo executing Question 1. The output shows the model correctly translating the query into SPARQL and returning the appropriate URI.

6 Critical Reflection

6.1 KB Quality Impact & Noise Issues

The most persistent challenge encountered during this project stemmed from the Wikidata expansion phase. Adding 70,000 predicate-controlled triples successfully expanded the dataset's mathematical volume, but effectively polluted the graph with maintenance metadata. Forcing an aggressive filtering step prior to KGE training resulted in a dataset so sparse that advanced embedding models failed completely.

6.2 Rule-Based vs. Embedding-Based Reasoning

The experiments underscored a classic tradeoff in semantic web technologies. SWRL rules provided flawless precision and logical transparency but felt rigid, requiring exhaustive manual configuration. In contrast, KGE models like TransE scaled effortlessly and mapped latent relational similarities, yet they provided no hard logical guarantees and proved overly sensitive to data sparsity.

6.3 SPARQL Generation Limitations and Improvements

Evaluating the Mistral-driven RAG system revealed that converting natural language to SPARQL relies on more than just syntax correction. Three critical failure modes became apparent:

- **Negative Constraint Blindness:** Even though the system prompt explicitly commanded the model to *never* hardcode Wikidata IDs, Mistral ignored this rule in Question 4 (`BIND(wd:"Q56")`). LLMs notoriously struggle with negative instructions.
- **Semantic Hallucinations:** While the self-repair loop successfully catches `rdflib` Python exceptions (like syntax errors), it is powerless against semantic hallucinations. In Question 3, the query was valid, but the model arbitrarily swapped "cast member" for "original language" (`P364`). It also invented nonexistent properties like `P910`.
- **Namespace Confusion:** In Question 2, the model unexpectedly injected a DBpedia URI into a Wikidata-based graph, highlighting how pre-training weights interfere with local RAG context.

If I were to rebuild this pipeline, I would abandon hardcoded schemas in the prompt. Instead, I would implement a dense vector-retrieval step to dynamically fetch the exact required

predicates based on the user's question, passing only the verified URIs to the LLM. Furthermore, swapping out wide `SELECT` expansions for precise `CONSTRUCT` queries would whitelist only cinema-relevant triples at the source, entirely neutralizing the metadata noise issue.